Exploratory Data Analysis in SQL

Chapter 1: What's in the Database?

DataCamp Course | Personal Study Notes & Lab Work

Author: Imon | 2026

This document covers Chapter 1 of the DataCamp course Exploratory Data Analysis in SQL. It includes key concepts, SQL queries written during exercises, dummy data, and query outputs to show exactly what happens when each query runs.

1. Exploring Table Sizes

Concept

The first step in EDA is understanding how large each table is. COUNT(*) counts every row in the table regardless of NULL values. It does not care about columns — it just counts how many rows exist.

Key Rules

- COUNT(*) = counts ALL rows including those with NULL values
- COUNT(column_name) = counts only rows where that column is NOT NULL
- The difference between the two = number of missing (NULL) values

SQL Query

```
-- Count total rows in fortune500 table
SELECT COUNT(*) AS total_rows
  FROM fortune500;
```

Dummy Data — fortune500 table

rank	company	ticker	revenues	revenues_change
1	Walmart	WMT	485,873	0.8
2	Berkshire Hathaway	BRKA	223,604	6.1
3	Apple	AAPL	215,639	-7.7
4	Exxon Mobil	XOM	205,004	-16.7
5	McKesson	MCK	198,015	3.4

Query Output

total_rows
500

2. Counting Missing Values

Concept

NULL values represent missing data. To count how many values are missing in a column, subtract the count of non-null values from the total row count.

Formula

```
Missing values = COUNT(*) - COUNT(column_name)
```

Why This Works

- COUNT(*) counts ALL rows — for example 500 rows total
- COUNT(ticker) skips NULLs — counts only rows with actual ticker values
- 500 - 468 = 32 missing ticker values

SQL Query

```
-- Count missing values in ticker column
SELECT COUNT(*) - COUNT(ticker) AS missing_ticker
  FROM fortune500;

-- Count missing in multiple columns at once
SELECT COUNT(*) - COUNT(ticker)   AS missing_ticker,
       COUNT(*) - COUNT(industry) AS missing_industry
  FROM fortune500;
```

Dummy Data

company	ticker	industry
Apple	AAPL	Technology
Ford	F	NULL
Nike	NULL	Retail
Tesla	TSLA	NULL
Google	GOOGL	Technology
Amazon	NULL	NULL

Query Output

missing_ticker	missing_industry
2	3

Interpretation: ticker has 2 missing (Nike, Amazon). industry has 3 missing (Ford, Tesla, Amazon).

3. Joining Tables

Concept

Tables are joined to combine related data from multiple tables. The common column used for matching must have compatible values across both tables. Just because two tables have a column with the same name does not mean the values are compatible.

All 4 JOIN Types

JOIN Type	What it keeps
INNER JOIN	Only rows that match in BOTH tables
LEFT JOIN	All rows from LEFT table + NULLs where no match in right
RIGHT JOIN	All rows from RIGHT table + NULLs where no match in left
FULL JOIN	ALL rows from BOTH tables — NULLs where no match either side

SQL Query — INNER JOIN

```
-- Join company and fortune500 on ticker column
SELECT company.name
  FROM company
       INNER JOIN fortune500
       ON company.ticker = fortune500.ticker;
```

Dummy Data

company table:

id	ticker	name
1	AAPL	Apple
2	WMT	Walmart
3	PYPL	PayPal
4	TSLA	Tesla

fortune500 table:

ticker	revenue
AAPL	215,639
WMT	485,873
GOOGL	110,855
AMZN	135,987

Query Output — INNER JOIN (only matches kept)

name
Apple
Walmart

PayPal and Tesla dropped — not in fortune500. Google and Amazon dropped — not in company table.

4. Primary Keys and Foreign Keys

Concept

Primary Key: A column that uniquely identifies each row. Every value must be unique and never NULL. Foreign Key: A column that references the primary key in another table, creating a formal link. Self Reference: A foreign key that points back to the same table.

Example — Self Reference (parent company)

id (PK)	name	parent_id (FK → id)
1	Volkswagen	NULL
2	Audi	1
3	Lamborghini	1
4	Audi Sport	2

parent_id references id in the same table. Audi (id=2) is a child of Volkswagen (id=1).

5. COALESCE Function

Concept

COALESCE returns the first non-NULL value from a list of arguments. It checks each argument left to right and returns the first one that is not NULL. Useful for providing default or backup values when a column contains NULLs.

Syntax

```
COALESCE(value1, value2, value3, ...)
```

How It Works

- COALESCE(NULL, 1, 2) = 1 — first non-NULL is 1
- COALESCE(NULL, NULL) = NULL — all are NULL so returns NULL
- COALESCE(2, 3, NULL) = 2 — first value is already non-NULL

SQL Query

```
-- Fill missing industry with sector, then 'Unknown'
SELECT COALESCE(industry, sector, 'Unknown') AS industry2,
       COUNT(*) AS count
  FROM fortune500
 GROUP BY industry2
 ORDER BY count DESC
 LIMIT 1;
```

Dummy Data

company	industry	sector
Apple	Technology	Technology
Walmart	Retail	Retailing
Company X	NULL	Finance
Company Y	NULL	Healthcare
Company Z	NULL	NULL

After COALESCE(industry, sector, 'Unknown')

company	industry2	which value was used?
Apple	Technology	industry (not NULL)
Walmart	Retail	industry (not NULL)
Company X	Finance	sector (industry was NULL)
Company Y	Healthcare	sector (industry was NULL)
Company Z	Unknown	'Unknown' (both were NULL)

6. Casting Data Types

Concept

Casting converts data from one type to another. Information can be lost — for example casting a decimal number to integer drops the decimal part. There are two syntax options in PostgreSQL that do the same thing.

Two Syntax Options

```
-- Option 1: CAST function
SELECT CAST(profits_change AS integer);

-- Option 2: :: shortcut (PostgreSQL only)
```

```
SELECT profits_change::integer;

-- Both produce exactly the same result!
```

Key Data Types

Type	Description	Example
INTEGER	Whole numbers only — no decimals allowed	10, -5, 200
NUMERIC	Decimal numbers — keeps full precision	3.14, -7.5, 0.001
TEXT	Text or string values	'hello', 'AAPL'
BOOLEAN	True or false only	TRUE, FALSE

SQL Queries

```
-- 1. Cast profits_change to integer (loses decimal)
SELECT profits_change,
       CAST(profits_change AS integer) AS profits_change_int
  FROM fortune500;

-- 2. Integer vs Numeric division
SELECT 10/3           AS integer_division,
       10::numeric/3  AS numeric_division;

-- 3. Cast text to numeric
SELECT '3.2'::numeric,
       '-123'::numeric,
       '1e3'::numeric,
       '1e-3'::numeric;
```

Query Outputs

profits_change casting — decimal is cut off when casting to integer:

profits_change	profits_change_int
10.7	10
-3.2	-3
5.9	5
-14.4	-14

Integer vs Numeric division:

integer_division	numeric_division
3	3.3333333333333333

Text to numeric — 1e3 is scientific notation = 1 x 10^3 = 1000:

'3.2'	'-123'	'1e3'	'1e-3'
3.2	-123	1000	0.001

7. Summarizing Distribution of Numeric Values

Concept

To understand how a numeric column is distributed across rows, we group values together and count how many rows fall into each group. Casting to integer first allows us to group similar decimal values into the same bucket.

SQL Execution Order — Critical to Understand!

Step	Clause	What it does
1	FROM	Gets all rows from the table
2	WHERE	Filters rows based on a condition
3	GROUP BY	Groups rows with the same value together and counts them
4	SELECT	Chooses which columns to show in the result
5	ORDER BY	Sorts the final result
6	LIMIT	Restricts how many rows are shown

SQL Queries

```
-- Distribution of revenue changes
SELECT revenues_change::integer AS change_bucket,
       COUNT(*) AS count
  FROM fortune500
 GROUP BY revenues_change::integer
 ORDER BY revenues_change::integer;

-- Why cast in BOTH SELECT and GROUP BY?
-- GROUP BY runs BEFORE SELECT
-- Without ::integer in GROUP BY:
--   10.7, 10.2, 10.9 each get their own group (useless!)
-- With ::integer in GROUP BY:
--   10.7, 10.2, 10.9 all become 10 = same group (useful!)

-- Count companies where revenue increased
SELECT COUNT(*) AS companies_increased
  FROM fortune500
 WHERE revenues_change > 0;
```

Dummy Data

company	revenues_change
Walmart	0.8
Berkshire	6.1
Apple	-7.7
Exxon	-16.7
Nike	0.9
Google	6.5
Amazon	-7.2

Distribution Query Output

revenues_change (integer)	count	interpretation
-16	1	Exxon — revenue dropped ~16%
-7	2	Apple + Amazon — dropped ~7%
0	2	Walmart + Nike — barely changed
6	2	Berkshire + Google — grew ~6%

Companies with Revenue Increase (WHERE revenues_change > 0)

companies_increased
3

Walmart (0.8), Berkshire (6.1) and Nike (0.9) had positive revenue change in 2017.

Chapter 1 — Key Takeaways

- COUNT(*) counts all rows. COUNT(column) skips NULLs.
- Missing values = COUNT(*) - COUNT(column_name)
- INNER JOIN keeps only rows matching in BOTH tables.
- LEFT JOIN keeps all left table rows — NULLs where no right match.
- Primary Key = unique row identifier. Foreign Key = reference to another table.
- Self Reference = foreign key pointing to the same table (e.g. parent company).
- COALESCE returns first non-NULL value — great for filling missing data.
- CAST or :: converts data types. Decimals are lost when casting to INTEGER.
- Numeric division keeps decimals. Integer division cuts them off.
- SQL execution order: FROM → WHERE → GROUP BY → SELECT → ORDER BY → LIMIT